

Annexe – TP1

1 Implémentation d'une application client/serveur

1.1 Fonctions de base

1.1.1 Includes pour les primitives réseau

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <arpa/inet.h>
```

1.1.2 Gestion de l'endianess

Les protocoles de base de l'Internet sont en **big-endian**. Pour écrire des programmes portables :

```
uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);
```

h = host, **n** = network

1.1.3 Gestion des SAP

Un SAP est défini par le type `sockaddr`. Comme le montre sa définition ci-dessous, c'est un type opaque.

```
struct sockaddr {
    unsigned short sa_family; // AF_XXX
    char sa_data[14]; // adresse protocolaire
};
```

Dans ce cours, on utilise UDP ou TCP/IPv4 nos SAP sont définis par le type `sockaddr_in` :

```
struct in_addr {
    unsigned int s_addr; // IP (big-endian)
};

struct sockaddr_in {
    unsigned short sin_family; // AF_INET
    unsigned short sin_port; // port TCP/UDP
    struct in_addr sin_addr; // IP
    char padding[8];
};
```

On peut passer de `sockaddr*` à `sockaddr_in*` par simple cast.

1.1.4 getaddrinfo

La fonction `getaddrinfo` initialise une liste de `addrinfo` (et donc de SAP) en fonction de :

- **node** : le serveur (nom humain ou dotted format)
- **service** : le port (un nombre (1200) ou un nom (http))
- **hints->ai_flags** : `AI_PASSIVE` pour serveur TCP, 0 pour les autres
- **hints->ai_family** : spécifie la couche réseau : `AF_INET` (IPv4)

- **hints->ai_socktype** : spécifie la couche transport :
 - SOCK_STREAM (TCP)
 - SOCK_DGRAM (UDP)
- **hints->ai_...** : 0

Si on obtient plusieurs SAP, on prend le premier.

```
int getaddrinfo(const char *node,
               const char *service,
               const struct addrinfo *hints,
               struct addrinfo **res);

void freeaddrinfo(struct addrinfo *res);

const char *gai_strerror(int errcode);
```

```
struct addrinfo {
    int ai_flags;
    int ai_family;
    int ai_socktype;
    int ai_protocol;
    socklen_t ai_addrlen;
    struct sockaddr *ai_addr;
    char *ai_canonname;
    struct addrinfo *ai_next;
};
```

Exercice Écrivez la fonction :

```
void getTCPSap(struct sockaddr* sap,
              socklen_t *saplen,
              const char *host,
              const char *port);
```

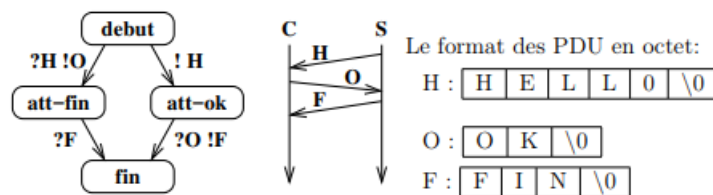
Si **host** est non nul, elle initialise ***sap** pour un client TCP/IPv4 sur le serveur **host:port**.

Si **host** est nul, elle initialise ***sap** pour un serveur TCP/IPv4.

En cas d'échec, elle affiche un message d'erreur et termine le processus avec un statut de 1.

1.2 Le protocole HELLO

1.2.1 Automate



1.2.2 PDU

```
H : HELLO\0
O : OK\0
F : FIN\0
```

1.3 Client TCP

1.3.1 Initialisation

```
const char *prgname;

int main(int argc, char **argv)
{
    prgname = argv[0];
    int statut;

    /* vérification des arguments */
    if (argc != 3) {
        fprintf(stderr, "%s : usage %s serveur port\n",
            prgname, prgname);
        exit(1);
    }

    char *namesvr = argv[1];
    char *service = argv[2];
```

1.3.2 Connexion au serveur

```
/* création du SAP du serveur */
struct sockaddr svrSAP;
socklen_t svrSAPlen;

getTCPsap(...., ...., ...., ....);

/* connexion au serveur */
int cx;

if ((cx = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
    fprintf(stderr, "%s : pb socket : %s\n",
        argv[0], strerror(errno));
    exit(1);
}

if (connect(cx, ....., .....) == -1) {
    fprintf(stderr, "%s : pb connect : %s\n",
        argv[0], strerror(errno));
    exit(1);
}
```

Remarque : cx est un flux FIFO bidirectionnel (comme un pipe)

cx est un double flux d'octets FIFO, un du client vers le serveur, l'autre du serveur vers le clients. Ils se comportent comme un pipe. Le nombre de lecteurs est et le nombre d'écrivains est Une lecture est si la FIFO est vide et qu'il y a encore Une lecture renvoie EOF si la FIFO est et qu'il n'y a plus Une écriture est si la FIFO est pleine et qu'il y a encore Une écriture renvoie une erreur si il n'y a plus Dans ce cas le signal est envoyé à

1.3.3 Dialogue

```
/* Dialogue état début */
char PDU[100];

statut = lire_PDU(PDU, cx);
if (statut != 'H') goto error;
```

```

printf("client : reçu \"%s\" ; ", PDU);

sprintf(PDU, "OK");
write(cx, ...., ....);

printf("envoyé \"%s\"\n", PDU);

/* Dialogue état att-fin */
statut = lire_PDU(PDU, cx);
if (statut != 'F') goto error;

printf("client : reçu \"%s\" ; quitte", PDU);

```

Important : TCP = flux d'octets → nécessite lire_PDU

cx est un flux d'octets qui contient les PDU les uns derrière les autres. De plus ils peuvent être mal formatés. Il faut donc une routine pour extraire un PDU du flux d'octets et les vérifier. C'est le rôle de "lire_PDU". Définissez et écrivez la fonction lire_PDU.

1.3.4 Terminaison

```

/* Terminaison état fin */
close(cx);
return 0;

error:
fprintf(stderr, "%s : message de type %c est inattendu\n",
        prgname, statut);
close(cx);
return 1;
}

```

1.4 Serveur TCP (naïf)

1.4.1 Initialisation

```

const char *prgname;

int main(int argc, char **argv)
{
    prgname = argv[0];
    int statut;

    /* vérification des arguments */
    if (argc != 2) {
        fprintf(stderr, "%s : usage %s port\n",
                prgname, prgname);
        exit(1);
    }

    char *service = argv[1];

```

1.4.2 Attachement aux clients

```

/* création du SAP des clients */
struct sockaddr cltsSAP;
socklen_t cltsSAPlen;

getTCPSap(...., ...., ...., ....);

/* activer l'automate de connexion */

```

```

int sock;

if ((sock = socket(..., ..., ...)) == -1) {
    fprintf(stderr, "%s : pb socket : %s\n", argv[0], strerror(errno));
    exit(1);
}

if (bind(sock, &cltsSAP, cltsSAPlen) < 0) {
    fprintf(stderr, "%s : pb bind %s\n", argv[0], strerror(errno));
    exit(1);
}

if (listen(sock, 100) != 0) {
    fprintf(stderr, "%s : pb listen %s\n", argv[0], strerror(errno));
    exit(1);
}

```

1.4.3 Boucle infinie du serveur

Connexion à un client

```

while (1) {

    int cx;
    struct sockaddr cltSAP;
    socklen_t cltSAPlen = sizeof(cltSAP);

    /* création du flux de communication (cx) */
    if ((cx = accept(sock, &cltSAP, &cltSAPlen)) == -1) {
        fprintf(stderr, "%s : pb accept %s\n",
            argv[0], strerror(errno));
        exit(1);
    }

    /* Dialogue état début */
    ...

    /* Dialogue état att-ok */
    ...

    /* Terminaison du dialogue (état fin) */
    ...
}

```

Dialogue

```

/* Dialogue état début */
char PDU[100];

sprintf(PDU, "HELLO");
write(cx, PDU, 6);

printf("serveur : envoie \"%s\"\n", PDU);

/* Dialogue état att-ok */
statut = lire_PDU(PDU, cx);
if (statut != 'O') goto error;

printf("serveur : recu \"%s\" ; ", PDU);

sprintf(PDU, "FIN");
write(cx, PDU, 4);

printf("envoie \"%s\" ; quitte\n", PDU);

```

Terminaison du dialogue

```
while (1) {

    int cx;
    struct sockaddr cltSAP;
    socklen_t cltSAPlen = sizeof(cltSAP);

    /* creation du flux de communication (cx) */
    if ((cx = accept(sock, &cltSAP, &cltSAPlen)) == -1) {
        fprintf(stderr, "%s : pb accept : %s\n",
            prgname, strerror(errno));
        exit(1);
    }

    /* Dialogue état début */
    close(cx);

    /* Terminaison du dialogue */

    continue;

error:
    close(cx); fprintf(stderr, "prgname, statut)

;
```

1.5 Résumé

socket	création socket
bind	attacher port
listen	attente connexions
accept	accepter client
connect	connexion client
read/write	communication
close	fermeture

	clt	svr	fonction
s'attacher aux clients ou au serveur	*	*	getaddrinfo(...) -> sap & saplen
	cx=	s=	socket(AF_INET, SOCK_STREAM, 0)
		*	bind(s, sap, saplen)
		*	listen(s, 100)
connexion	*		connect(cx, sap, saplen)
		cx=	cx = accept(s, &clt, &cltlen)
dialogue	*	*	read(cx, data, len)
	*	*	write(cx, data, len)
fermer la connexion	*	*	close(cx)

bind : 1) réserve le SAP (un autre bind sur le même SAP échouera) ; 2) attache le SAP à la socket.

listen : active l'automate de connexion TCP pour le SAP attaché à la socket. Le N indique le nombre maximal d'automates de connexion qui peuvent tourner en même temps.